UNIT 5.GRAPHICAL USER INTERFACE (GUI) IN PYTHON

Definition:
A Graphical User Interface (GUI) in Python allows users to interact with a program using graphical elements like buttons, windows, and menus instead of text commands.

Features of GUI

- User-friendly interface
- Uses windows, buttons, labels
- Event-driven programming
- Improves user interaction

Popular GUI Libraries in Python

- **Tkinter** (built-in)
- PyQt
- Kivy
- wxPython

Tkinter Basics

Tkinter is the standard GUI library in Python.

Simple GUI Example

```
import tkinter as tk

root = tk.Tk()
root.title("My First GUI")

label = tk.Label(root, text="Hello World")
label.pack()

root.mainloop()
```

Common Widgets

- Label → display text
- Button → perform action
- Entry → input field
- Frame → container

Button Example

```
import tkinter as tk

def click():
    print("Button clicked")

root = tk.Tk()

btn = tk.Button(root, text="Click Me", command=click)
btn.pack()

root.mainloop()
```

Graphical User Interface (GUI) in Python

Introduction

A Graphical User Interface (GUI) allows users to interact with software through visual elements such as windows, icons, and buttons. Python provides several libraries to develop GUI applications easily.

Definition

GUI is a visual interface that enables interaction between the user and the computer system using graphical components.

GUI Libraries in Python

- **Tkinter** → Built-in and widely used
- **PyQt** → Advanced GUI applications
- **Kivy** → Touch-based applications
- **wxPython** → Native look and feel

Tkinter Components (Widgets)

Widget	Purpose
Label	Displays text
Button	Performs action
Entry	User input
Frame	Container
Text	Multi-line input

Basic Structure of Tkinter Program

```
import tkinter as tk

# Create window
root = tk.Tk()
root.title("GUI Example")

# Add widget
label = tk.Label(root, text="Welcome to GUI")
label.pack()

# Run application
root.mainloop()
```

Event Handling

GUI works on events like button clicks.

```
def show():
    print("Clicked")

btn = tk.Button(root, text="Click", command=show)
```

Advantages of GUI

- Easy to use
- Attractive interface
- No need to remember commands
- Improves user experience

Disadvantages

- More memory usage
- Slower than command-line interface

- Complex to design

✅ ROOT WINDOW IN PYTHON

Definition:
The root window is the main window of a GUI application created using Tkinter. It acts as the base for all other widgets.

Creating a Root Window

```
import tkinter as tk

root = tk.Tk()
root.mainloop()
```

Features of Root Window

- Main window of application
- Holds all widgets
- Controls program execution
- Required for GUI applications

Properties of Root Window

1. Title

```
root.title("My App")
```

---

2. Size (Geometry)

```
root.geometry("400x300")
```

---

3. Background Color

```
root.configure(bg="lightblue")
```

---

4. Resizable

```
root.resizable(False, False)
```

Example Program

```
import tkinter as tk

root = tk.Tk()
root.title("Sample Window")
root.geometry("300x200")

label = tk.Label(root, text="Hello GUI")
label.pack()

root.mainloop()
```

Root Window in Python (Tkinter)

Introduction

In Python GUI programming using Tkinter, the root window is the first and main window created. It serves as the container for all GUI components such as buttons, labels, and input fields.

Definition

The root window is the main application window created using `Tk()` which acts as the base for all widgets in a GUI program.

Creating Root Window

```
import tkinter as tk
root = tk.Tk()
```

Common Properties of Root Window

1. Title

Sets the window title.

```
root.title("My Application")
```

---

2. Geometry

Defines window size.

```
root.geometry("500x400")
```

---

3. Background Color

```
root.configure(bg="white")
```

---

4. Resizable

Controls resizing.

```
root.resizable(True, True)
```

---

5. Icon

```
root.iconbitmap("icon.ico")
```

Adding Widgets to Root Window

```
label = tk.Label(root, text="Welcome")
label.pack()
```

Main Loop

```
root.mainloop()
```

✓ Keeps the window running
✓ Waits for user interaction

Advantages

- Central control of GUI
- Easy to manage widgets
- Essential for GUI creation

✅ FONTS AND COLOURS IN PYTHON (Tkinter)

Fonts: Used to change the style, size, and appearance of text in GUI.
Colours: Used to set foreground (text) and background of widgets.

Font in Tkinter

Fonts define how text appears.

Example:

```
import tkinter as tk

root = tk.Tk()
label = tk.Label(root, text="Hello", font=("Arial", 14))
label.pack()
root.mainloop()
```

Colours in Tkinter

Used to make GUI attractive.

```
label = tk.Label(root, text="Hello", fg="blue", bg="yellow")
```

- `fg` → text color
- `bg` → background color

Font Styles

```
label = tk.Label(root, text="Text",
                 font=("Arial", 16, "bold"))
```

Font Options:

- Font family (Arial, Times)
- Size (12, 14, 18)
- Style (bold, italic, underline)

Colour Options

```
root.configure(bg="lightblue")
```

Types:

- Named colors → "red", "green"
- Hex colors → "#FF5733"

Example Program

```
import tkinter as tk

root = tk.Tk()
root.geometry("300x200")

label = tk.Label(root, text="Welcome",
                 font=("Arial", 16, "bold"),
                 fg="white", bg="blue")
label.pack()

root.mainloop()
```

Fonts and Colours in Python GUI (Tkinter)

Introduction

Fonts and colours play an important role in designing attractive and user-friendly GUI applications. Tkinter provides various options to customize text appearance and background colors.

Fonts in Tkinter

Definition

Fonts control the appearance of text such as size, style, and typeface.

Syntax

```
font=("font_name", size, "style")
```

Example

```
label = tk.Label(root, text="Hello",
                 font=("Times New Roman", 18, "italic"))
```

Font Styles

- Normal
- Bold
- Italic
- Underline

Colours in Tkinter

Definition

Colours are used to enhance the visual appearance of GUI components.

Foreground and Background

```
fg="red"   # Text color
bg="yellow"  # Background color
```

Types of Colours

1. Named colours → "red", "blue", "green"
2. Hexadecimal colours → "#RRGGBB"

Example Program

```
import tkinter as tk

root = tk.Tk()
root.title("Fonts and Colours")

root.configure(bg="lightgrey")

label = tk.Label(root,
                 text="Python GUI",
                 font=("Arial", 20, "bold"),
                 fg="white",
                 bg="black")
```

```
label.pack(pady=20)

root.mainloop()
```

Advantages

- Improves user experience
- Makes GUI attractive
- Enhances readability

Definition:
Containers are widgets used to hold and organize other widgets in a GUI application.

Types of Containers in Tkinter

- **Frame** → Basic container
- **LabelFrame** → Container with title
- **PanedWindow** → Resizable container
- **TopLevel** → New window

Example (Frame)

```
import tkinter as tk

root = tk.Tk()

frame = tk.Frame(root)
frame.pack()

btn = tk.Button(frame, text="Click")
btn.pack()

root.mainloop()
```

Common Containers

1. Frame

Groups widgets together.

```
frame = tk.Frame(root, bg="lightblue")
frame.pack()
```

---

2. LabelFrame

Container with label.

```
lf = tk.LabelFrame(root, text="User Info")
lf.pack()
```

---

3. PanedWindow

Splits window into resizable sections.

```
pw = tk.PanedWindow(root)
pw.pack()
```

---

4. TopLevel

Creates a new window.

```
top = tk.Toplevel(root)
```

Containers in Python GUI (Tkinter)

Introduction

Containers in Tkinter are special widgets used to organize and manage other widgets in a structured way. They help in designing clean and user-friendly interfaces.

Definition

A container is a widget that holds and arranges other widgets within a GUI application.

Types of Containers

1. Frame

A simple container used to group widgets.

```
frame = tk.Frame(root, bg="lightgrey")
frame.pack()
```

---

2. LabelFrame

A frame with a title.

```
lf = tk.LabelFrame(root, text="Login")
lf.pack()
```

---

3. PanedWindow

Divides window into sections.

```
pw = tk.PanedWindow(root)
pw.pack()
```

---

4. TopLevel

Creates a separate window.

```
top = tk.Toplevel(root)
```

Example Program

```
import tkinter as tk

root = tk.Tk()
root.title("Containers Example")

frame = tk.Frame(root, bg="lightblue")
frame.pack(pady=10)

label = tk.Label(frame, text="Inside Frame")
label.pack()

lf = tk.LabelFrame(root, text="Options")
lf.pack(pady=10)

btn = tk.Button(lf, text="Click Me")
btn.pack()

root.mainloop()
```

Advantages

- Organizes layout properly
- Improves GUI structure
- Makes design flexible
- Easy to manage widgets

✅ CANVAS IN PYTHON (Tkinter)

Definition:
Canvas is a Tkinter widget used to draw shapes, graphics, images, and text in a GUI application.

Features of Canvas

- Used for drawing graphics
- Supports shapes like lines, rectangles, circles
- Can display text and images
- Useful for games and animations

Creating a Canvas

```
import tkinter as tk

root = tk.Tk()

canvas = tk.Canvas(root, width=300, height=200)
canvas.pack()

root.mainloop()
```

Drawing Shapes on Canvas

1. Line

```
canvas.create_line(0, 0, 200, 100)
```

---

2. Rectangle

```
canvas.create_rectangle(50, 50, 150, 100)
```

---

3. Oval (Circle)

```
canvas.create_oval(50, 50, 150, 150)
```

---

4. Text

```
canvas.create_text(100, 50, text="Hello")
```

Example Program

```
import tkinter as tk

root = tk.Tk()

canvas = tk.Canvas(root, width=300, height=200)
canvas.pack()

canvas.create_line(0, 0, 200, 100)
canvas.create_rectangle(50, 50, 150, 100)

root.mainloop()
```

Canvas in Python (Tkinter)

Introduction

The Canvas widget in Tkinter is used to create and manipulate graphical elements in a GUI application. It allows drawing shapes, displaying text, and creating interactive graphics.

Definition

Canvas is a widget that provides a drawing area where shapes, images, and text can be displayed.

Creating a Canvas

```
canvas = tk.Canvas(root, width=400, height=300, bg="white")
canvas.pack()
```

Drawing Methods

1. Line

```
canvas.create_line(x1, y1, x2, y2)
```

---

2. Rectangle

```
canvas.create_rectangle(x1, y1, x2, y2)
```

---

3. Oval (Circle)

```
canvas.create_oval(x1, y1, x2, y2)
```

---

4. Text

```
canvas.create_text(x, y, text="Sample")
```

---

5. Image

```
canvas.create_image(x, y, image=img)
```

Example Program

```
import tkinter as tk

root = tk.Tk()
root.title("Canvas Example")

canvas = tk.Canvas(root, width=400, height=300, bg="lightyellow")
canvas.pack()

canvas.create_line(0, 0, 200, 100)
canvas.create_rectangle(50, 50, 150, 120)
canvas.create_oval(200, 50, 300, 150)
canvas.create_text(200, 200, text="Hello Canvas")

root.mainloop()
```

Advantages

- Supports graphics and drawing
- Useful for games and animations
- Flexible and interactive